



Sichuan University -
Pittsburgh Institute

ECE0202 Embedded Processors & Interfacing + Lab

Sichuan University-Pittsburgh Institute



Function Call in C

```
int bar(){
```

```
    A=A+1;
```

```
    foo();
```

```
    B=B+1;
```

```
}
```

```
void foo(){
```

```
    C=C+1;
```

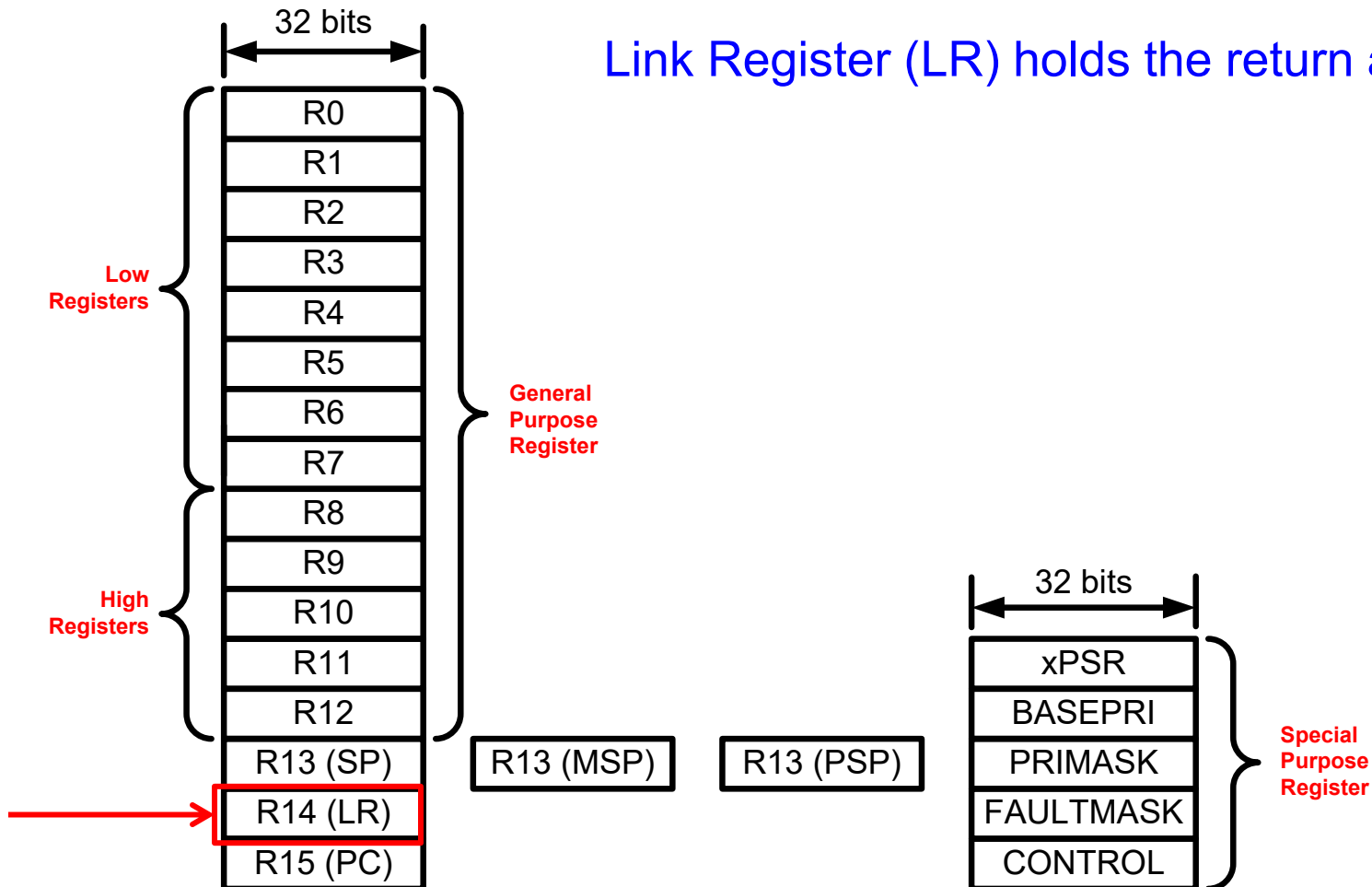
```
}
```

Caller

Callee/subroutine

Link Register

Link Register (LR) holds the return address



Calling a Subroutine

BL *label*

- Step 1: $LR = PC + 4$
- Step 2: $PC = \text{label}$
- Notes:
 - *label* is name of subroutine
 - Compiler translates label to memory address
 - After call, LR holds return address (the instruction following the call)

Caller Program

```
MOV r4, #100
...
BL  foo
...
```

Subroutine/Callee

```
foo PROC
...
MOV    r4, #10
...
BX     LR
ENDP
```

Exiting a Subroutine

BX LR

- $PC = LR$

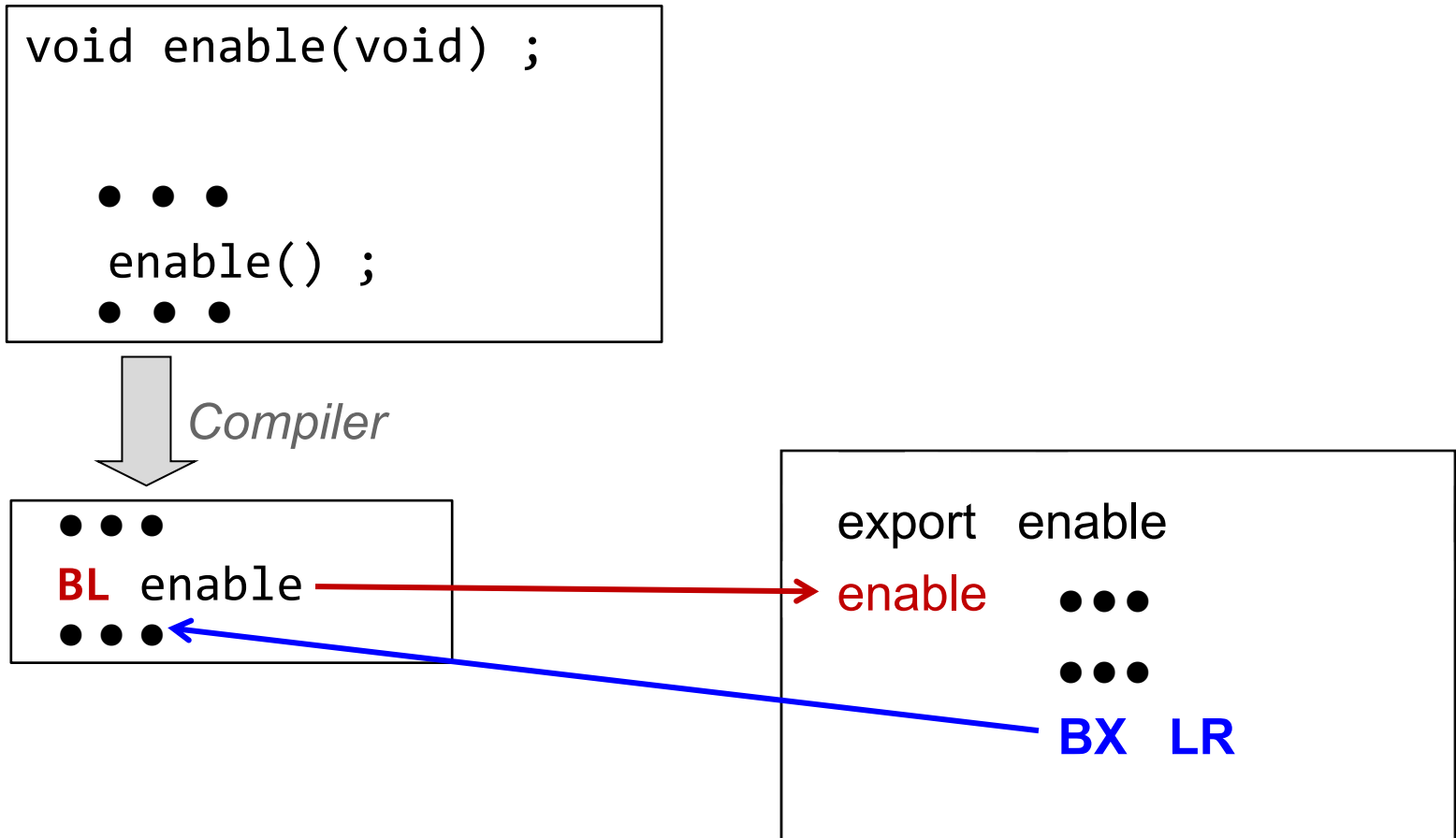
Caller Program

```
MOV r4, #100  
...  
BL  foo  
...
```

Subroutine/Callee

```
foo PROC  
...  
MOV    r4, #10  
...  
BX    LR  
ENDP
```

BL and BX



Function Call in C

```
void bar(){
```

```
  A=A+1;
```

```
  foo();
```

```
  B=B+1;
```

```
}
```

```
void foo(){
```

```
  C=C+1;
```

```
}
```

Caller

Callee/subroutine

Function Call in C

```
void bar(){  
    A=A+1;  
    B = foo(A);  
    B=B+1;  
}  
  
int foo(int C){  
    C=C+1;  
    return C;  
}
```

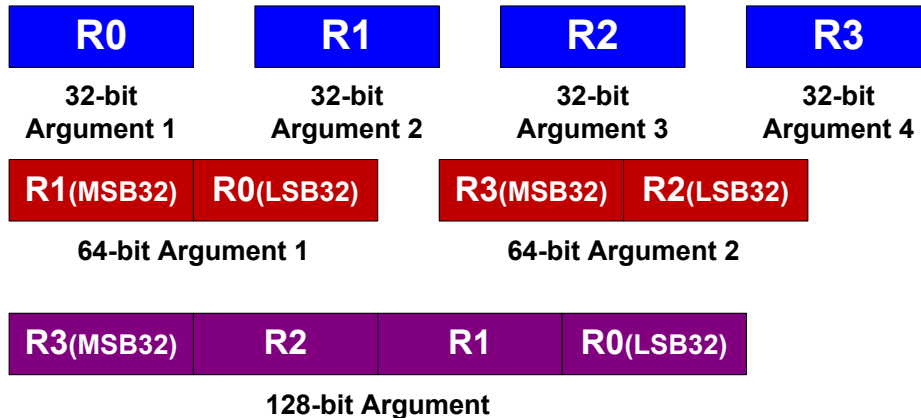
Caller

Callee/subroutine

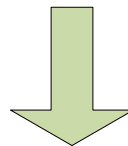
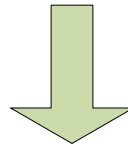
ARM Procedure Call Standard

Register	Usage	Subroutine Preserved	Notes
r0	Argument 1 and return value	No	If return has 64 bits, then r0:r1 hold it. If argument 1 has 64 bits, r0:r1 hold it.
r1	Argument 2	No	
r2	Argument 3	No	If the return has 128 bits, r0-r3 hold it.
r3	Argument 4	No	If more than 4 arguments, use the stack
r4	General-purpose V1	Yes	Variable register 1 holds a local variable.
r5	General-purpose V2	Yes	Variable register 2 holds a local variable.
r6	General-purpose V3	Yes	Variable register 3 holds a local variable.
r7	General-purpose V4	Yes	Variable register 4 holds a local variable.
r8	General-purpose V5	YES	Variable register 5 holds a local variable.
r9	Platform specific/V6	No	Usage is platform-dependent.
r10	General-purpose V7	Yes	Variable register 7 holds a local variable.
r11	General-purpose V8	Yes	Variable register 8 holds a local variable.
r12 (IP)	Intra-procedure-call register	No	It holds intermediate values between a procedure and the sub-procedure it calls.
r13 (SP)	Stack pointer	Yes	SP has to be the same after a subroutine has completed.
r14 (LR)	Link register	No	LR does not have to contain the same value after a subroutine has completed.
r15 (PC)	Program counter	N/A	Do not directly change PC

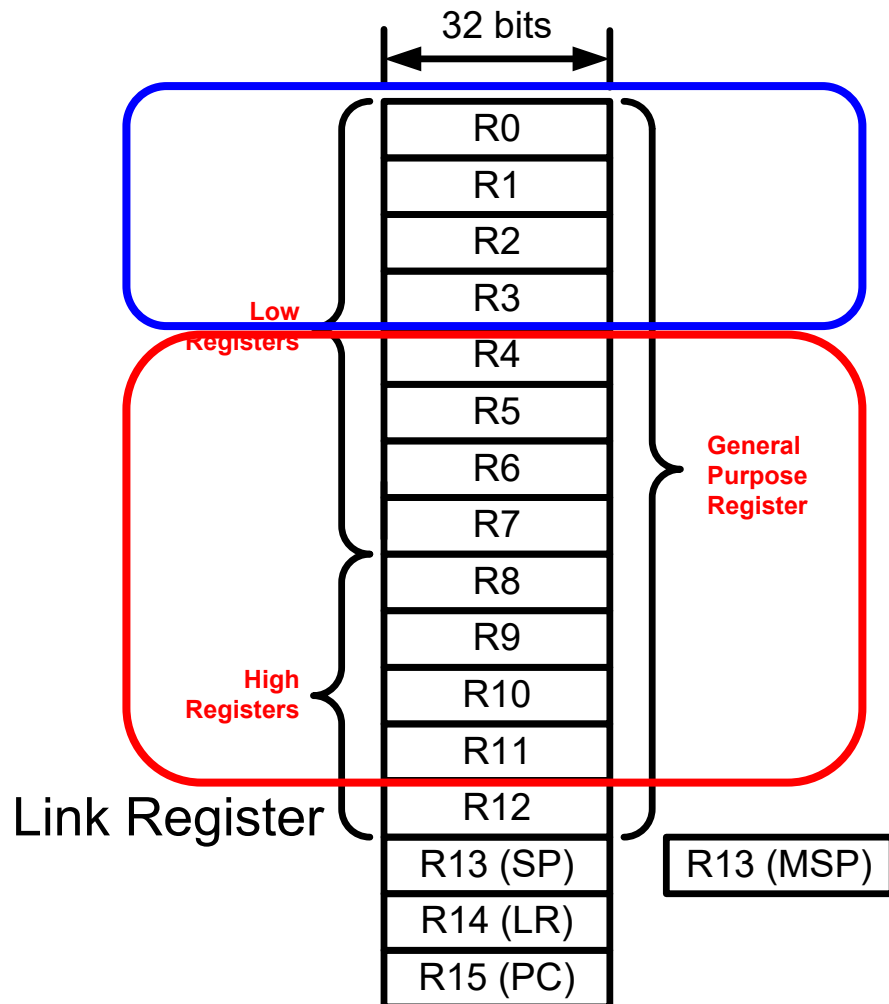
Passing Arguments via Registers



Extra arguments are pushed to the stack by the caller. The caller is responsible to pop them out of the stack after the subroutine returns.

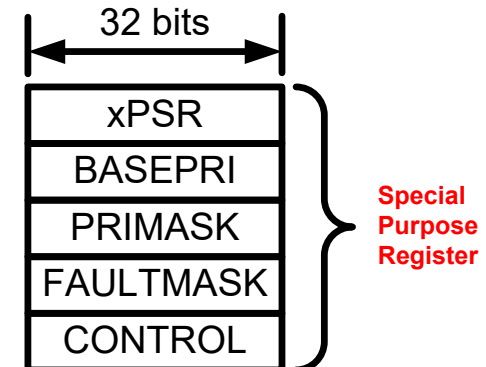


Link Register



Not saved. Hold arguments, results, or temporary values. Caller doesn't expect them to be retained.

Callee must save them. Caller expects these values are retained.



Example: $R2 = R0 * R0 + R1 * R1$

```

MOV R0, #3
MOV R1, #4
BL  SSQ
MOV R2, R0
B  ENDL

...
SSQ  PROC
    MUL R2, R0, R0
    MUL R3, R1, R1
    ADD R2, R2, R3
    MOV R0, R2
    BX  LR
    ENDP
...

```

R1: second argument

R0: first argument

```

int SSQ(int x, int y){
    int z;
    z = x*x + y * y;
    return z;
}

```

R0: Return Value

Example: $R2 = R0 * R0 + R1 * R1$

```

MOV R0,#3
MOV R1,#4
BL SSQ
MOV R2,R0
B ENDL

...
SSQ PROC
MUL R2,R0,R0
MUL R3,R1,R1
ADD R2,R2,R3
MOV R0,R2
BX LR
ENDP
ENDL ...

```

R0	
R1	
R2	
R3	
LR	
PC	0x08000128

SSQ

In fact, PC is 0x08000129 because bit 0 of PC should always be 1 for ARM Cortex-M to indicate thumb mode.

	Memory Address
MOV R0,#3	0x08000128
MOV R1,#4	0x0800012C
BL SSQ	0x08000130
MOV R2,R0	0x08000134
B ENDL	0x08000136
MUL R2,...	0x0800013B
MUL R3,...	0x0800013C
ADD R2,R3	0x08000140
MOV R0,R2	0x08000142
BX LR	0x08000144

Example: $R2 = R0 * R0 + R1 * R1$

```

MOV R0,#3
MOV R1,#4
BL SSQ
MOV R2,R0
B ENDL

...
SSQ PROC
MUL R2,R0,R0
MUL R3,R1,R1
ADD R2,R2,R3
MOV R0,R2
BX LR
ENDP
ENDL ...
    
```

R0	3
R1	
R2	
R3	
LR	
PC	0x08000128

SSQ

Memory Address	
0x08000128	MOV R0,#3
0x0800012C	MOV R1,#4
0x08000130	BL SSQ
0x08000134	MOV R2,R0
0x08000136	B ENDL
0x0800013B	MUL R2,...
0x0800013C	MUL R3,...
0x08000140	ADD R2,R3
0x08000142	MOV R0,R2
0x08000144	BX LR

Example: $R2 = R0 * R0 + R1 * R1$

```

MOV R0,#3
MOV R1,#4
BL SSQ
MOV R2,R0
B ENDL

...
PROC
SSQ  MUL R2,R0,R0
    MUL R3,R1,R1
    ADD R2,R2,R3
    MOV R0,R2
    BX LR
    ENDP
ENDL ...
    
```

R0	3
R1	4
R2	
R3	
LR	
PC	0x0800012C

SSQ

Memory Address	
0x08000128	MOV R0,#3
0x0800012C	MOV R1,#4
0x08000130	BL SSQ
0x08000134	MOV R2,R0
0x08000136	B ENDL
0x0800013B	MUL R2,...
0x0800013C	MUL R3,...
0x08000140	ADD R2,R3
0x08000142	MOV R0,R2
0x08000144	BX LR

Example: $R2 = R0 * R0 + R1 * R1$

```

MOV R0,#3
MOV R1,#4
BL SSQ
MOV R2,R0
B ENDL

...
PROC
MUL R2,R0,R0
MUL R3,R1,R1
ADD R2,R2,R3
MOV R0,R2
BX LR
ENDP
ENDL ...

```

R0	3
R1	4
R2	
R3	
LR	
PC	0x08000130

	Memory Address
MOV R0,#3	0x08000128
MOV R1,#4	0x0800012C
BL SSQ	0x08000130
MOV R2,R0	0x08000134
B ENDL	0x08000136
MUL R2,...	0x0800013B
MUL R3,...	0x0800013C
ADD R2,R3	0x08000140
MOV R0,R2	0x08000142
BX LR	0x08000144

SSQ

Example: $R2 = R0 * R0 + R1 * R1$

```

MOV R0,#3
MOV R1,#4
BL SSQ
MOV R2,R0
B ENDL

...
PROC
MUL R2,R0,R0
MUL R3,R1,R1
ADD R2,R2,R3
MOV R0,R2
BX LR
ENDP

ENDL ...

```

R0	3
R1	4
R2	
R3	
LR	0x08000134
PC	0x0800013B

SSQ

In fact, LR is 0x08000135 because bit 0 of PC should always be 1 for ARM Cortex-M to indicate thumb mode.

	Memory Address
MOV R0,#3	0x08000128
MOV R1,#4	0x0800012C
BL SSQ	0x08000130
MOV R2,R0	0x08000134
B ENDL	0x08000136
MUL R2,...	0x0800013B
MUL R3,...	0x0800013C
ADD R2,R3	0x08000140
MOV R0,R2	0x08000142
BX LR	0x08000144

Address of the next instruction after the branch is saved into LR.

Example: $R2 = R0 * R0 + R1 * R1$

```

MOV R0,#3
MOV R1,#4
BL SSQ
MOV R2,R0
B ENDL

...
PROC
MUL R2,R0,R0
MUL R3,R1,R1
ADD R2,R2,R3
MOV R0,R2
BX LR
ENDP
ENDL ...
    
```

R0	3
R1	4
R2	9
R3	

LR	0x08000134
PC	0x0800013B

	Memory Address
MOV R0,#3	0x08000128
MOV R1,#4	0x0800012C
BL SSQ	0x08000130
MOV R2,R0	0x08000134
B ENDL	0x08000136
MUL R2,...	0x0800013B
MUL R3,...	0x0800013C
ADD R2,R3	0x08000140
MOV R0,R2	0x08000142
BX LR	0x08000144

Example: $R2 = R0 * R0 + R1 * R1$

```

MOV R0,#3
MOV R1,#4
BL SSQ
MOV R2,R0
B ENDL

...
PROC
MUL R2,R0,R0
MUL R3,R1,R1
ADD R2,R2,R3
MOV R0,R2
BX LR
ENDP
ENDL ...

```

R0	3
R1	4
R2	9
R3	16

LR	0x08000134
PC	0x0800013C

	Memory Address
MOV R0,#3	0x08000128
MOV R1,#4	0x0800012C
BL SSQ	0x08000130
MOV R2,R0	0x08000134
B ENDL	0x08000136
MUL R2,...	0x0800013B
MUL R3,...	0x0800013C
ADD R2,R3	0x08000140
MOV R0,R2	0x08000142
BX LR	0x08000144

SSQ

Example: $R2 = R0 * R0 + R1 * R1$

```

MOV R0,#3
MOV R1,#4
BL SSQ
MOV R2,R0
B ENDL

...
PROC
MUL R2,R0,R0
MUL R3,R1,R1
ADD R2,R2,R3
MOV R0,R2
BX LR
ENDP
ENDL ...

```

R0	3
R1	4
R2	25
R3	16

LR	0x08000134
PC	0x08000140

	Memory Address
MOV R0,#3	0x08000128
MOV R1,#4	0x0800012C
BL SSQ	0x08000130
MOV R2,R0	0x08000134
B ENDL	0x08000136
MUL R2,...	0x0800013B
MUL R3,...	0x0800013C
ADD R2,R3	0x08000140
MOV R0,R2	0x08000142
BX LR	0x08000144

SSQ

Example: $R2 = R0 * R0 + R1 * R1$

```

MOV R0,#3
MOV R1,#4
BL SSQ
MOV R2,R0
B ENDL

...
PROC
MUL R2,R0,R0
MUL R3,R1,R1
ADD R2,R2,R3
MOV R0,R2
BX LR
ENDP
ENDL ...

```

R0	25
R1	4
R2	25
R3	16
LR	0x08000134
PC	0x08000142

	Memory Address
MOV R0,#3	0x08000128
MOV R1,#4	0x0800012C
BL SSQ	0x08000130
MOV R2,R0	0x08000134
B ENDL	0x08000136
MUL R2,...	0x0800013B
MUL R3,...	0x0800013C
ADD R2,R3	0x08000140
MOV R0,R2	0x08000142
BX LR	0x08000144

SSQ

Example: $R2 = R0 * R0 + R1 * R1$

```

MOV R0,#3
MOV R1,#4
BL SSQ
MOV R2,R0
B ENDL

...
PROC
MUL R2,R0,R0
MUL R3,R1,R1
ADD R2,R2,R3
MOV R0,R2
BX LR
ENDP

ENDL ...
    
```

R0	25
R1	4
R2	25
R3	16

LR	0x08000134
PC	0x08000144

	Memory Address
MOV R0,#3	0x08000128
MOV R1,#4	0x0800012C
BL SSQ	0x08000130
MOV R2,R0	0x08000134
B ENDL	0x08000136
MUL R2,...	0x0800013B
MUL R3,...	0x0800013C
ADD R2,R3	0x08000140
MOV R0,R2	0x08000142
BX LR	0x08000144

Example: $R2 = R0 * R0 + R1 * R1$

```

MOV R0,#3
MOV R1,#4
BL SSQ
MOV R2,R0
B ENDL

...
PROC
SSQ  MUL R2,R0,R0
    MUL R3,R1,R1
    ADD R2,R2,R3
    MOV R0,R2
    BX LR
    ENDP
ENDL ...

```

R0	25
R1	4
R2	25
R3	16

LR	0x08000134
PC	0x08000134

SSQ

Copy LR to PC when
returning from a subroutine!

	Memory Address
MOV R0,#3	0x08000128
MOV R1,#4	0x0800012C
BL SSQ	0x08000130
MOV R2,R0	0x08000134
B ENDL	0x08000136
MUL R2,...	0x0800013B
MUL R3,...	0x0800013C
ADD R2,R3	0x08000140
MOV R0,R2	0x08000142
BX LR	0x08000144

Example: $R2 = R0 * R0 + R1 * R1$

```

MOV R0,#3
MOV R1,#4
BL SSQ
MOV R2,R0
B ENDL

...
SSQ PROC
    MUL R2,R0,R0
    MUL R3,R1,R1
    ADD R2,R2,R3
    MOV R0,R2
    BX LR
    ENDP
ENDL ...
    
```

R0	25
R1	4
R2	25
R3	16
LR	0x08000134
PC	0x08000134

	Memory Address
MOV R0,#3	0x08000128
MOV R1,#4	0x0800012C
BL SSQ	0x08000130
MOV R2,R0	0x08000134
B ENDL	0x08000136
MUL R2,...	0x0800013B
MUL R3,...	0x0800013C
ADD R2,R3	0x08000140
MOV R0,R2	0x08000142
BX LR	0x08000144

SSQ

Example: $R2 = R0 * R0 + R1 * R1$

```

MOV R0,#3
MOV R1,#4
BL SSQ
MOV R2,R0
B ENDL

...
SSQ PROC
MUL R2,R0,R0
MUL R3,R1,R1
ADD R2,R2,R3
MOV R0,R2
BX LR
ENDP
ENDL ...

```

R0	25
R1	4
R2	25
R3	16

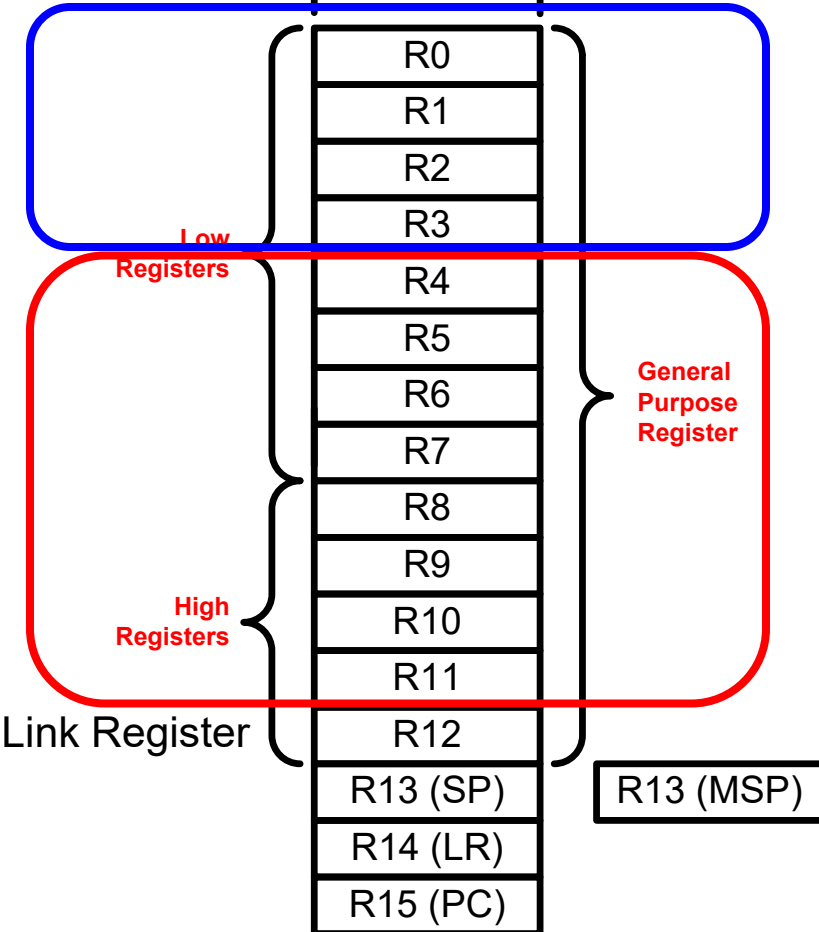
LR	0x08000134
PC	0x08000136

	Memory Address
MOV R0,#3	0x08000128
MOV R1,#4	0x0800012C
BL SSQ	0x08000130
MOV R2,R0	0x08000134
B ENDL	0x08000136
MUL R2,...	0x0800013B
MUL R3,...	0x0800013C
ADD R2,R3	0x08000140
MOV R0,R2	0x08000142
BX LR	0x08000144

SSQ

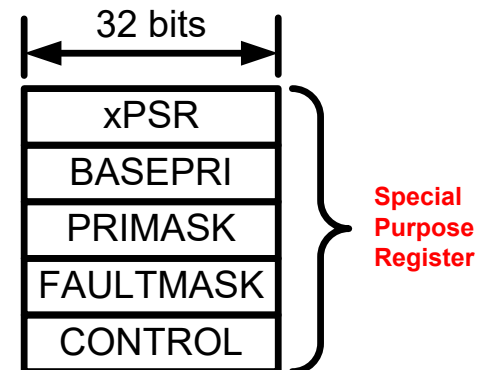
Link Register

32 bits



Scratch registers, not saved by subroutine. Hold arguments/result

Callee must save them.
Caller expects these values are retained.



Preserve Runtime Environment via Stack

Caller Program	Subroutine/Callee
<pre>MOV r4, #100 ... BL foo ... ADD r4, r4, #1 ; r4 = 101, not 11</pre>	<pre>foo PROC PUSH {r4} ; preserve r4 ... MOV r4, #10 ; foo changes r4 ... POP {r4} ; Recover r4 BX LR ENDP</pre>

Subroutine Calling Another Subroutine

MAIN

MOV R0, #2

BL QUAD

ENDL ...

Function **MAIN**

QUAD PUSH {LR}

BL SQ

BL SQ

POP {LR}

BX LR

Function **QUAD**

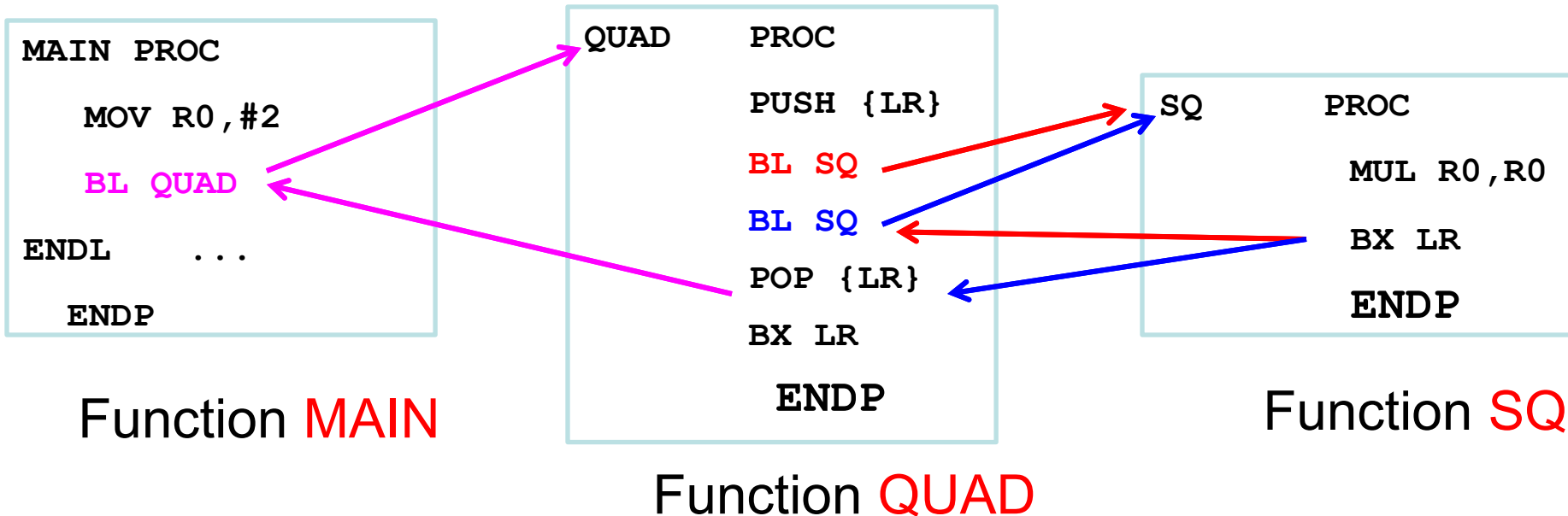
SQ

MUL R0, R0

BX LR

Function **SQ**

Subroutine Calling Another Subroutine



Example: $R_0 = R_0^4$

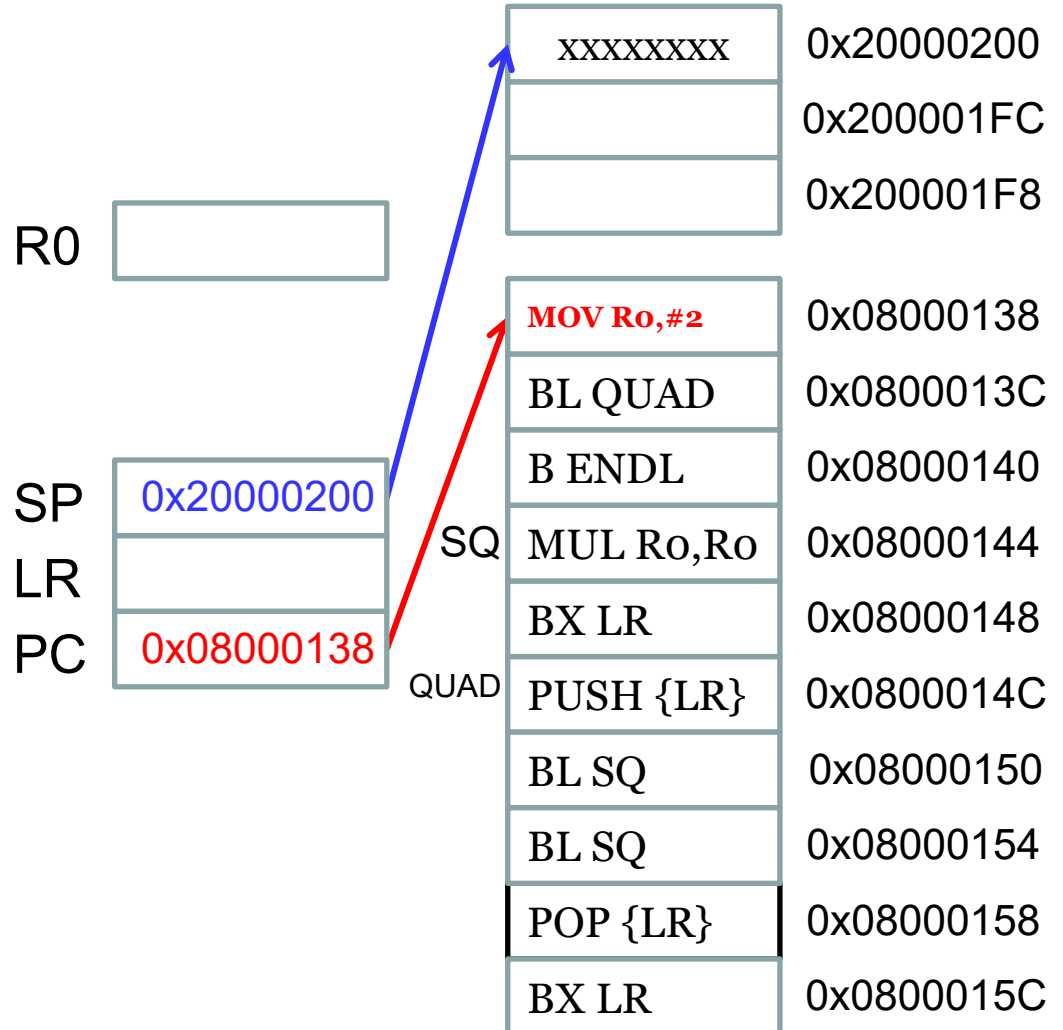
```

MOV R0, #2
BL QUAD
B ENDL

SQ      MUL R0, R0
        BX LR

QUAD    PUSH {LR}
        BL SQ
        BL SQ
        POP {LR}
        BX LR

ENDL    ...
    
```



Example: $R_0 = R_0^4$

```

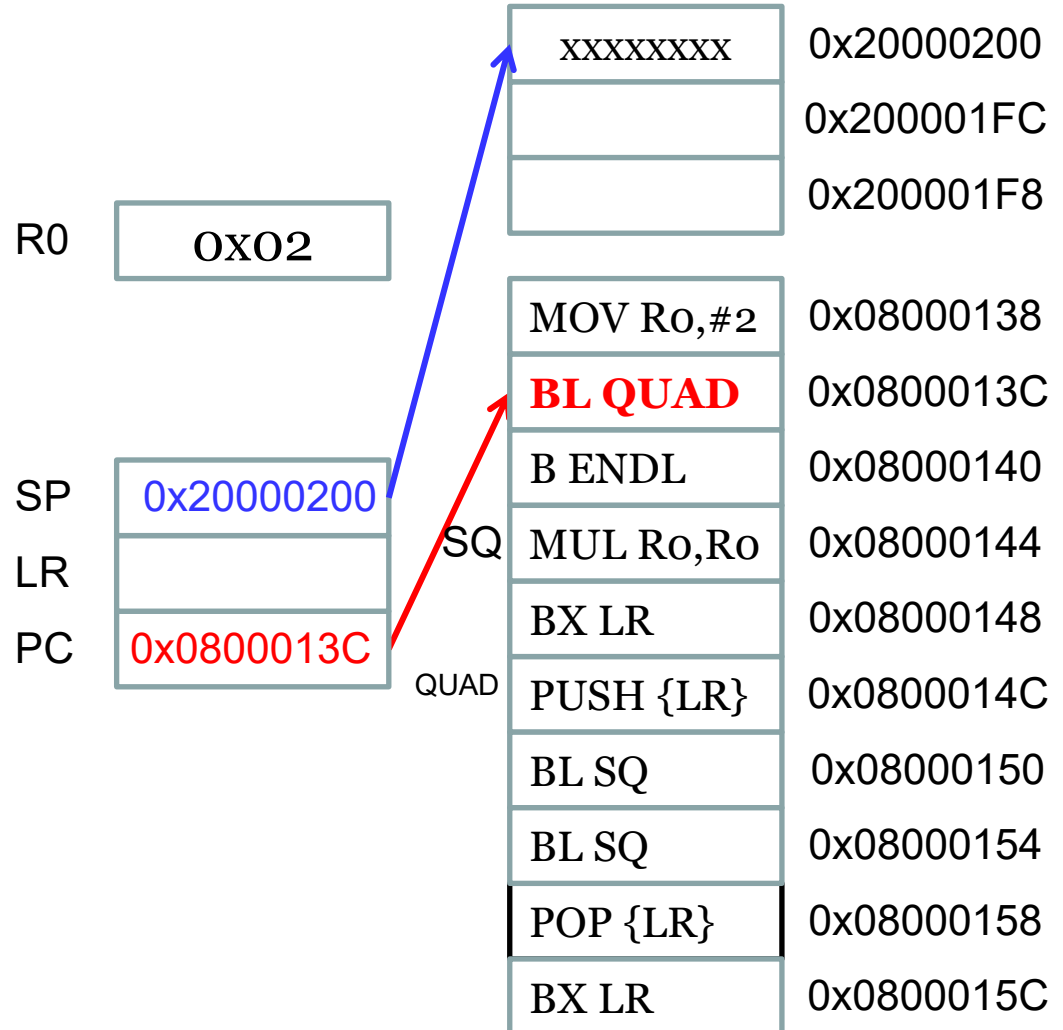
MOV R0,#2
BL QUAD
B ENDL

SQ      MUL R0,R0
        BX LR

QUAD    PUSH {LR}
        BL SQ
        BL SQ
        POP {LR}
        BX LR

ENDL    ...

```



Example: $R_0 = R_0^4$

```

MOV R0,#2
BL QUAD
B ENDL

SQ    MUL R0,R0
      BX LR

QUAD  PUSH {LR}
      BL SQ
      BL SQ
      POP {LR}
      BX LR

ENDL  ...
  
```

R0 0x02

SP 0x20000200
LR 0x08000140
PC 0x0800014C

Preserve
Link Register (LR)

XXXXXXXX	0x20000200
	0x200001FC
	0x200001F8
MOV R0,#2	0x08000138
BL QUAD	0x0800013C
B ENDL	0x08000140
SQ MUL R0,R0	0x08000144
BX LR	0x08000148
QUAD PUSH {LR}	0x0800014C
BL SQ	0x08000150
BL SQ	0x08000154
POP {LR}	0x08000158
BX LR	0x0800015C

Example: $R_0 = R_0^4$

```

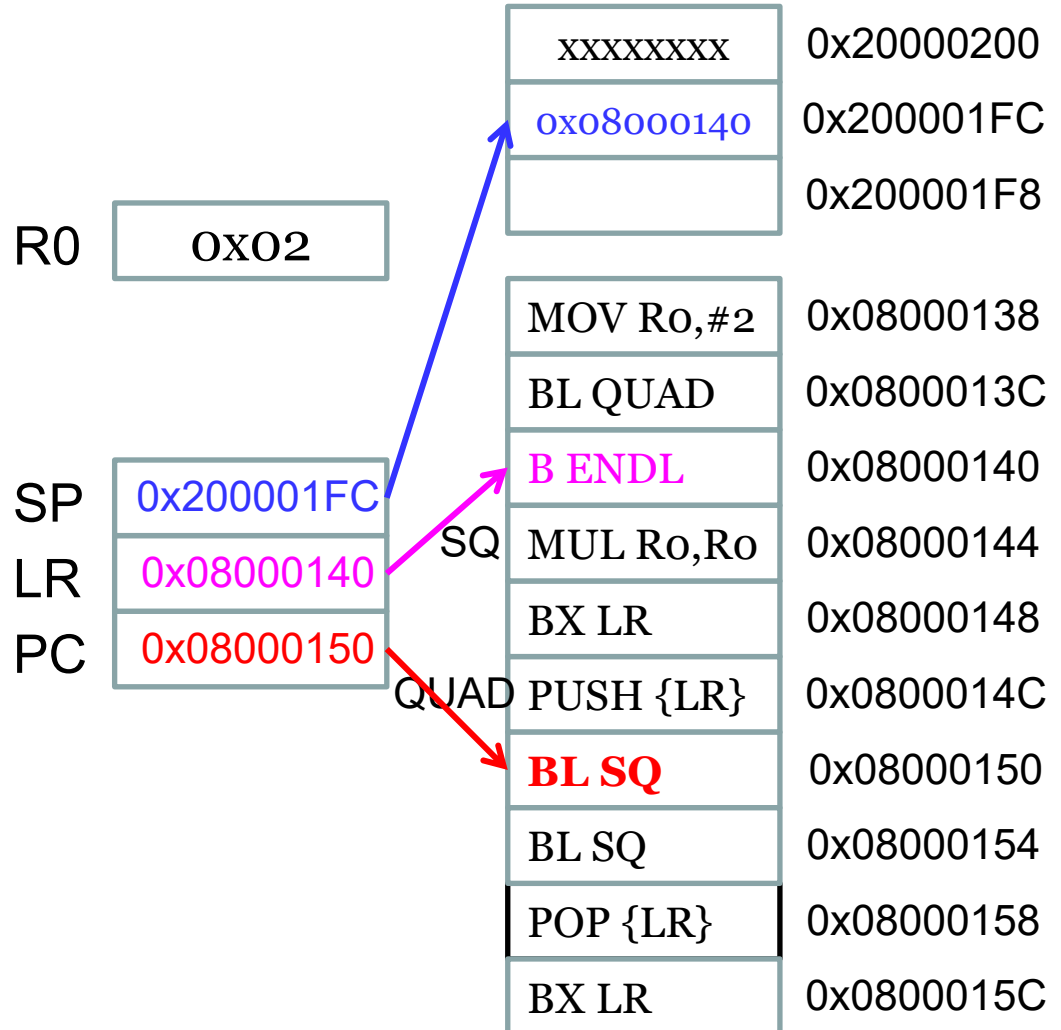
MOV R0,#2
BL QUAD
B ENDL

SQ      MUL R0,R0
        BX LR

QUAD    PUSH {LR}
        BL SQ
        BL SQ
        POP {LR}
        BX LR

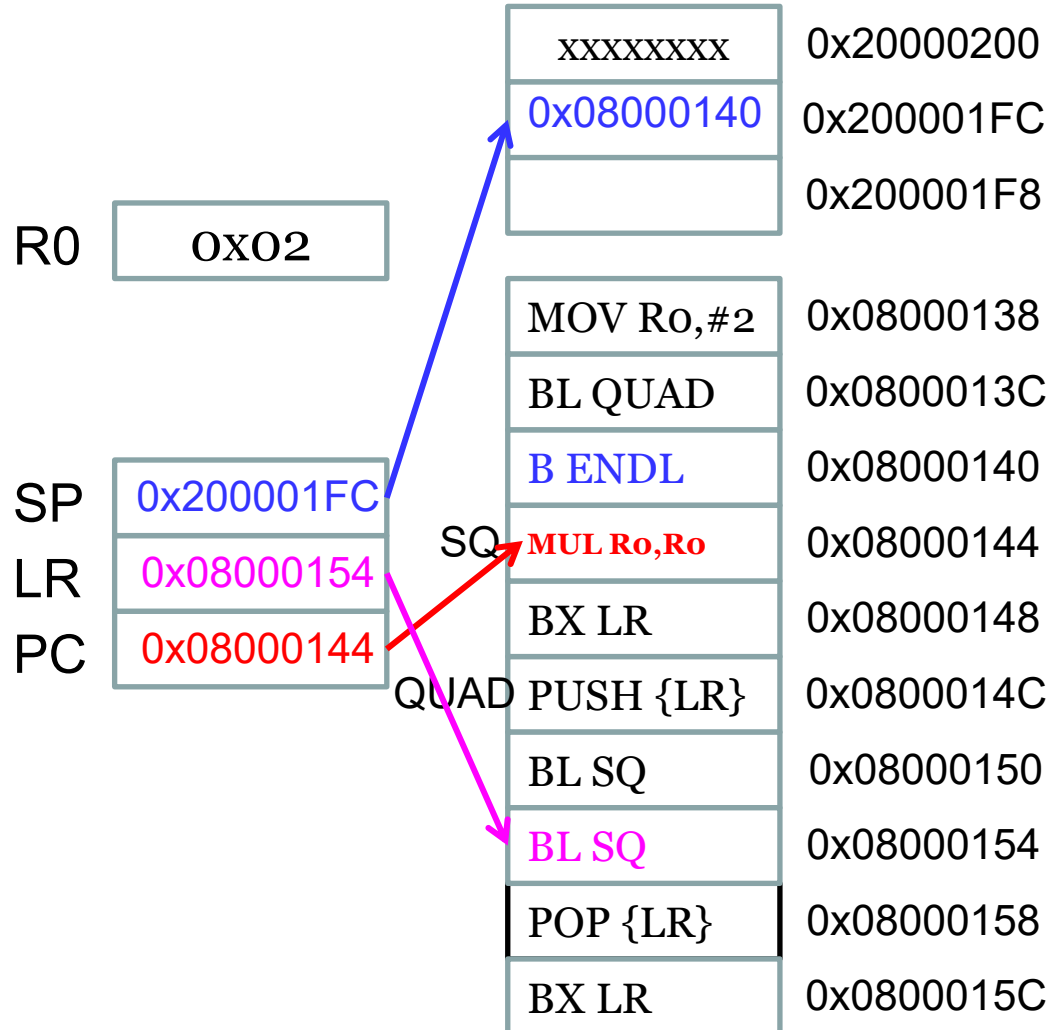
ENDL    ...

```



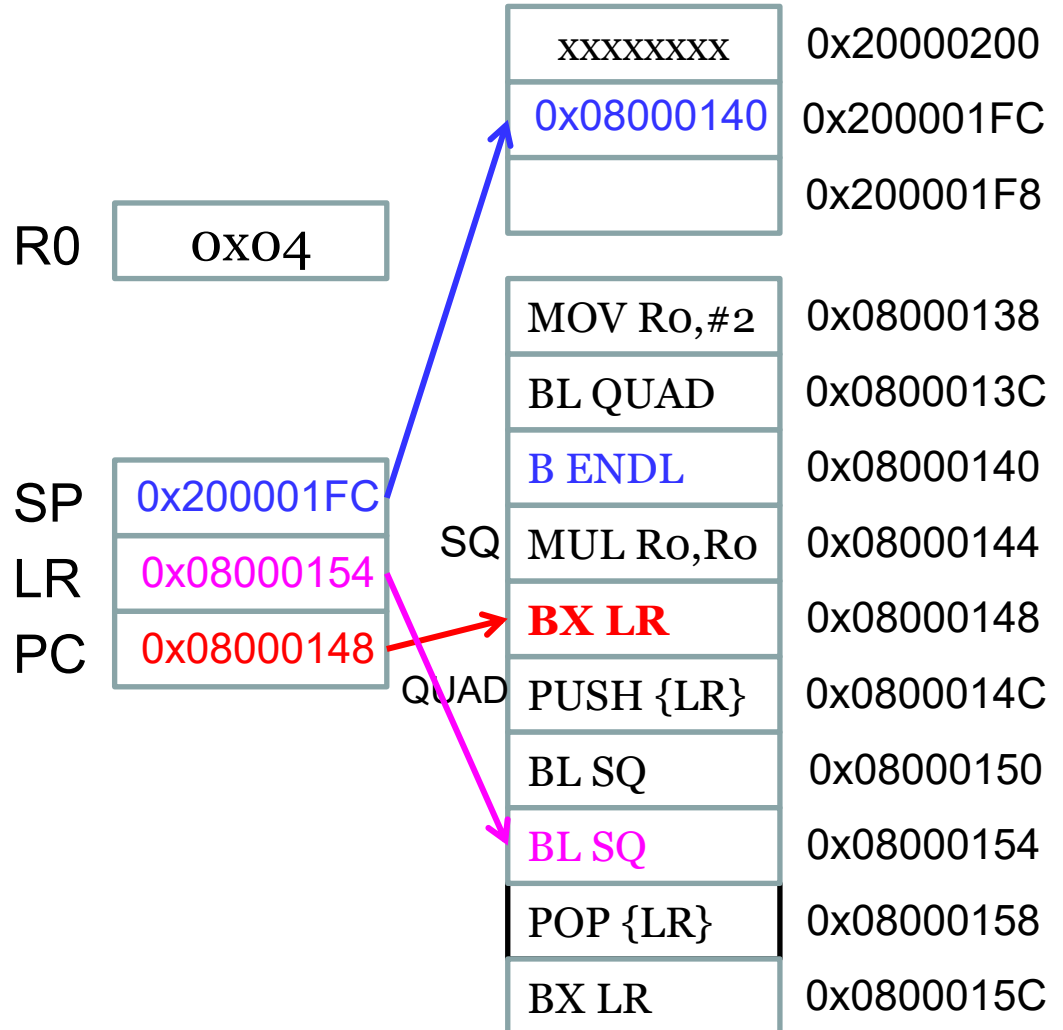
Example: $R_0 = R_0^4$

	MOV R0,#2
	BL QUAD
	B ENDL
SQ	MUL R0,R0
	BX LR
QUAD	PUSH {LR}
	BL SQ
	BL SQ
	POP {LR}
	BX LR
ENDL	...



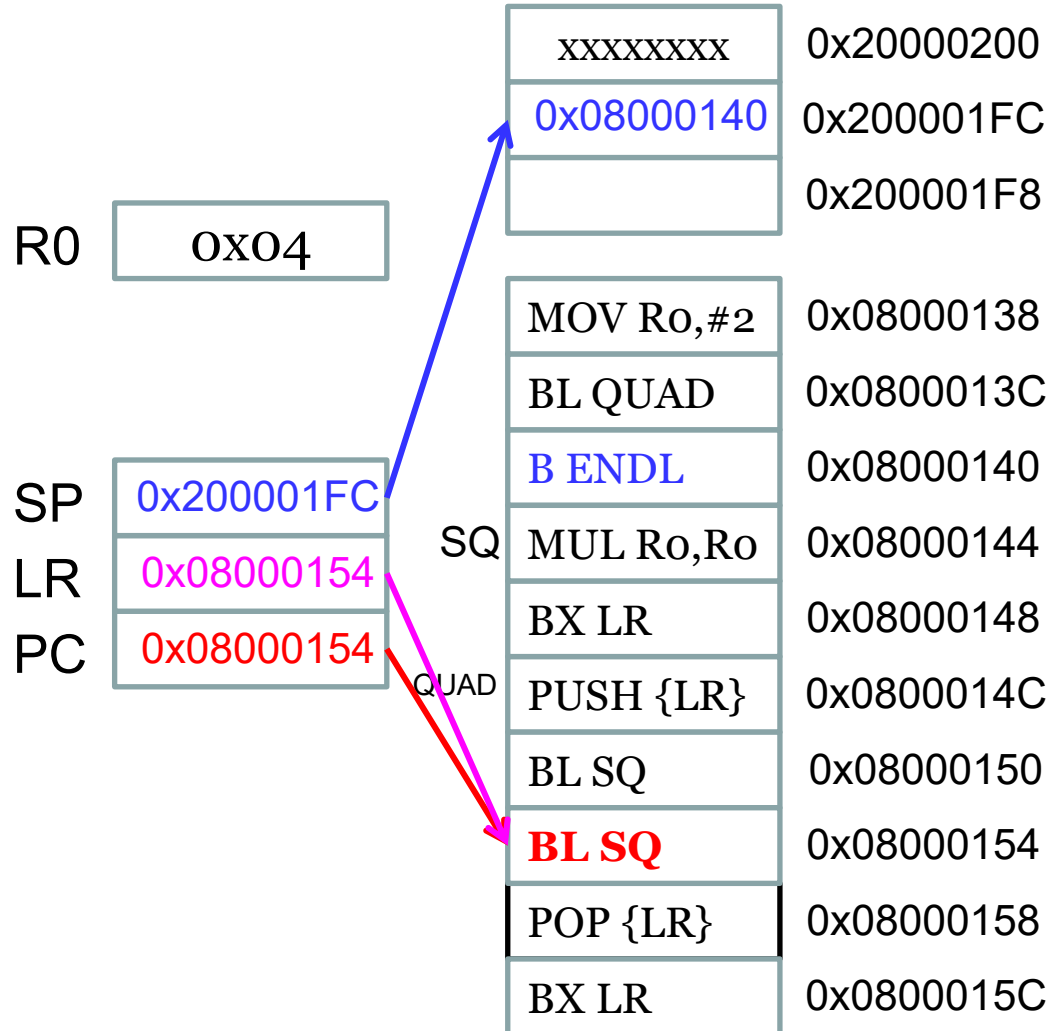
Example: $R_0 = R_0^4$

	MOV R0,#2
	BL QUAD
	B ENDL
SQ	MUL R0,R0
	BX LR
QUAD	PUSH {LR}
	BL SQ
	BL SQ
	POP {LR}
	BX LR
ENDL	...



Example: $R_0 = R_0^4$

	MOV R0,#2
	BL QUAD
	B ENDL
SQ	MUL R0,R0
	BX LR
QUAD	PUSH {LR}
	BL SQ
	BL SQ
	POP {LR}
	BX LR
ENDL	...



Example: $R_0 = R_0^4$

```

MOV R0,#2
BL QUAD
B ENDL

SQ      MUL R0,R0
BX LR

QUAD    PUSH {LR}
BL SQ
BL SQ
POP {LR}
BX LR

ENDL    ...

```

R0 0x04

SP 0x200001FC
LR 0x08000158
PC 0x08000144

XXXXXXXX	0x20000200
0x08000140	0x200001FC
	0x200001F8
MOV R0,#2	0x08000138
BL QUAD	0x0800013C
B ENDL	0x08000140
MUL R0,R0	0x08000144
BX LR	0x08000148
PUSH {LR}	0x0800014C
BL SQ	0x08000150
BL SQ	0x08000154
POP {LR}	0x08000158
BX LR	0x0800015C

SQ
QUAD

Example: $R_0 = R_0^4$

```

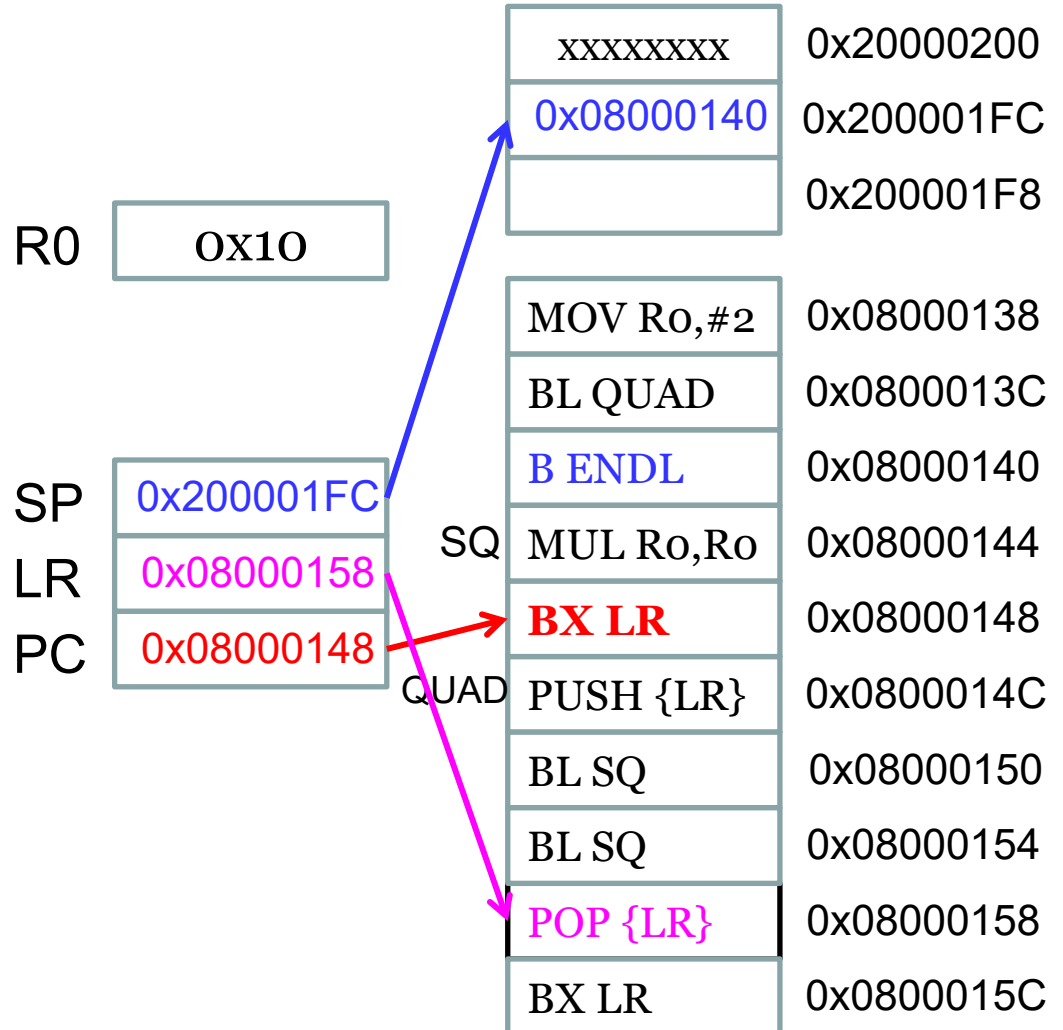
MOV R0,#2
BL QUAD
B ENDL

SQ      MUL R0,R0
        BX LR

QUAD    PUSH {LR}
        BL SQ
        BL SQ
        POP {LR}
        BX LR

ENDL    ...

```



Example: $R_0 = R_0^4$

```

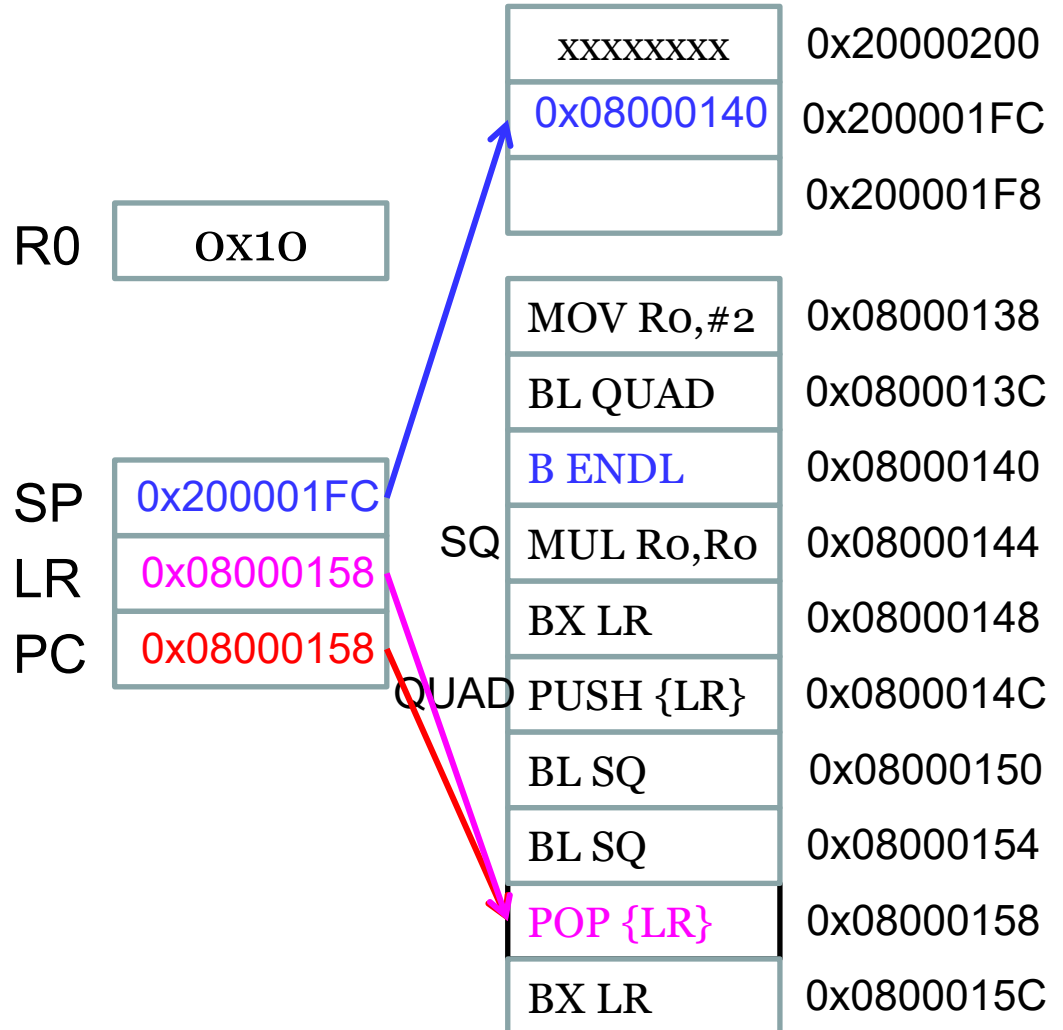
MOV R0,#2
BL QUAD
B ENDL

SQ      MUL R0,R0
        BX LR

QUAD    PUSH {LR}
        BL SQ
        BL SQ
        POP {LR}
        BX LR

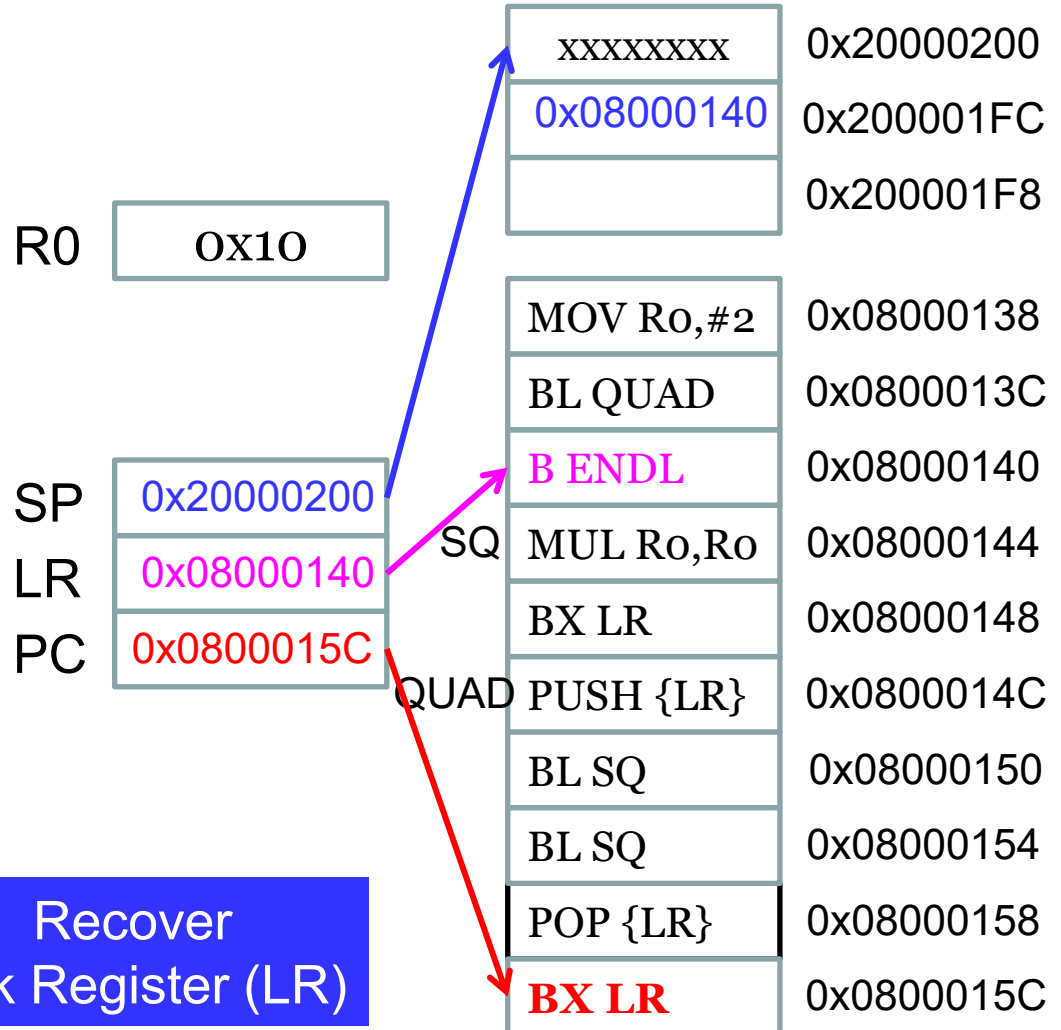
ENDL    ...

```



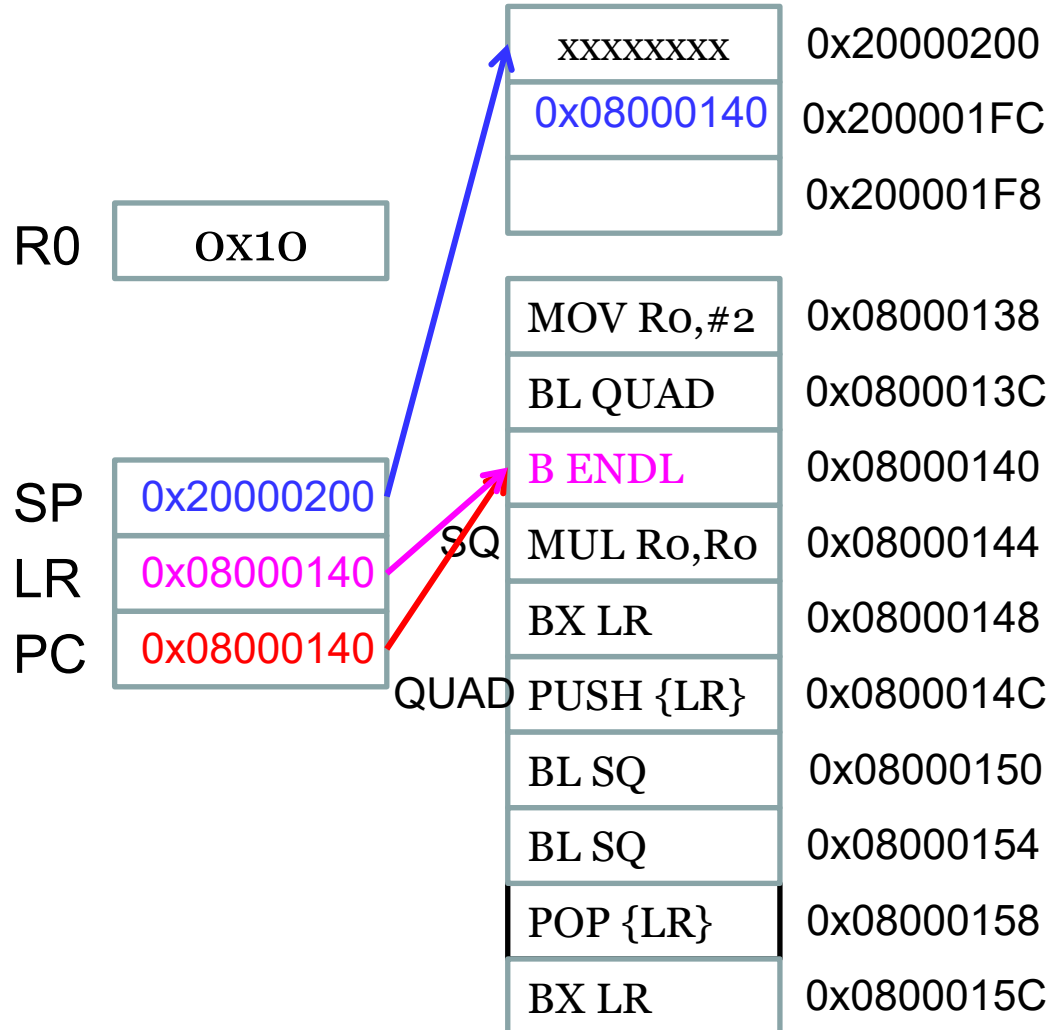
Example: $R_0 = R_0^4$

	MOV R0,#2
	BL QUAD
	B ENDL
SQ	MUL R0,R0
	BX LR
QUAD	PUSH {LR}
	BL SQ
	BL SQ
	POP {LR}
	BX LR
ENDL	...



Example: $R_0 = R_0^4$

	MOV R0,#2
	BL QUAD
	B ENDL
SQ	MUL R0,R0
	BX LR
QUAD	PUSH {LR}
	BL SQ
	BL SQ
	POP {LR}
	BX LR
ENDL	...



Recursive Functions

- A **recursive function** is one that solves its task by **calling itself** on smaller pieces of data.
- An effective tactic is to
 - divide a problem into sub-problems of the same type as the original,
 - solve those sub-problems, and
 - combine the results

Defining Factorial(n)

Product of the first n numbers

$$1 \times 2 \times 3 \times \dots \times n$$

$$\text{factorial}(0) = 1$$

$$\text{factorial}(1) = 1 \qquad = 1 \times \text{factorial}(0)$$

$$\text{factorial}(2) = 2 \times 1 \qquad = 2 \times \text{factorial}(1)$$

$$\text{factorial}(3) = 3 \times 2 \times 1 \qquad = 3 \times \text{factorial}(2)$$

$$\text{factorial}(4) = 4 \times 3 \times 2 \times 1 \qquad = 4 \times \text{factorial}(3)$$

$$\text{factorial}(n) = n \times (n-1) \times \dots \times 1 \qquad = n \times \text{factorial}(n-1)$$

Classic Example: Factorial

- Factorial is the classic example:
 - $6! = 6 \times 5!$
 - $6! = 6 \times 5 \times 4!$
 - ...
 - $6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1$
- The factorial function can be easily written as a recursive function:

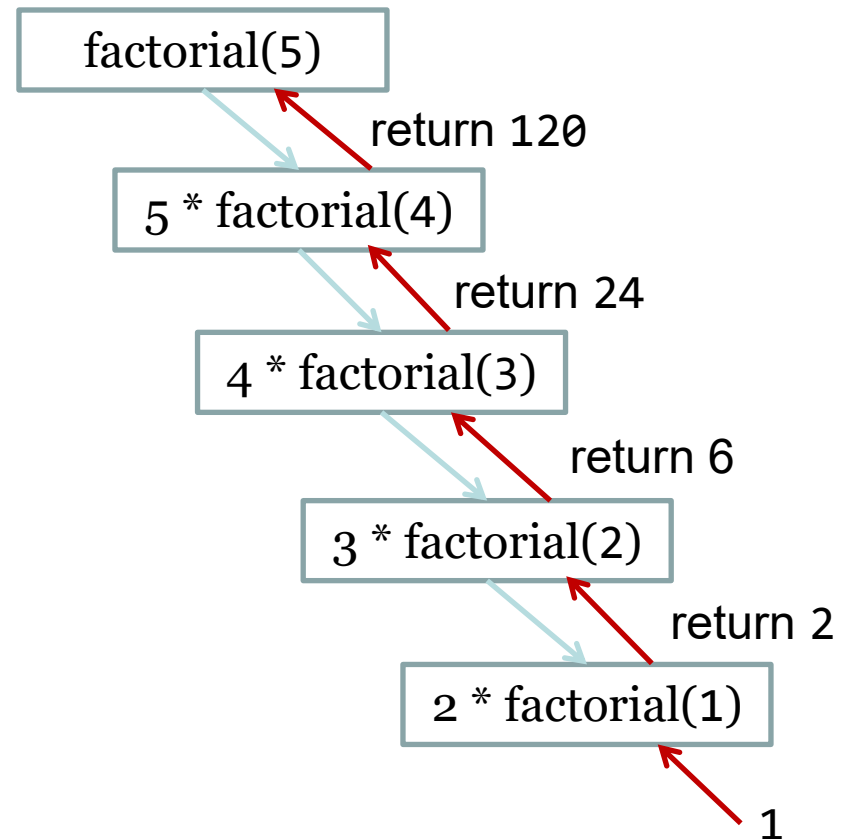
```
int Factorial(int n) {  
  
    if (n < 2)  
        return 1; /* base case */  
  
    return (n * Factorial(n - 1));  
  
}
```

Recursive Factorial in C

```
int factorial(int n);

int main(void){
    factorial(5);
    return 0;
}

int factorial(int n) {
    int f;
    if(n==1)
        return 1;
    else
        f = n * factorial(n-1);
    return f;
}
```



Recursive Functions

- **push LR** (& working registers) onto stack before nested call
- **pop LR** (& working registers) off stack after nested return

Recursive Factorial in Assembly

```
AREA    main, CODE, READONLY
EXPORT  __main
```

```
ENTRY
```

```
__main PROC
```

```
    MOV ro, #0x03
```

```
    BL  factorial
```

```
stop B stop
```

```
ENDP
```

factorial

```
PUSH {r4, lr}
```

```
MOV  r4, ro
```

```
CMP  r4, #0x01
```

```
BNE  NZ
```

```
MOVS ro, #0x01
```

```
loop POP {r4, pc}
```

```
NZ   SUBS ro, r4, #1
```

```
BL   factorial
```

```
MUL  ro, r4, ro
```

```
B    loop
```

```
END
```

